

Java Keywords

Java reserves about fifty words. A keyword cannot be used as the name of a variable, method or class, because the compiler already gives it a meaning.

We meet a small number of them every day. This covers those, grouped by what they do, with the ones we type most often first.

Access and object behavior → Classes and inheritance → Control flow
→ Exception handling → Types and checks → Threads and storage
→ Organizing code

1. Access and object behavior

These control who can see a member, and how it belongs to the class.

Keyword	What it does
<code>public</code>	Makes a member visible everywhere
<code>private</code>	Restricts a member to its own class
<code>protected</code>	Allows access within the package and subclasses
<i>(nothing)</i>	Restricts a member to the same package. Called package-private
<code>static</code>	Makes a member belong to the class rather than an object
<code>final</code>	Stops something from being changed
<code>new</code>	Creates an object

```

class Employee {
    public String name;           // anyone can read this
    private double salary;        // only this class can touch it
    protected String department; // this class, subclasses, same package

    static int count = 0;         // one copy for all employees
    final String company = "TechCorp"; // cannot be reassigned

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
        count++;
    }

    public double getSalary() {
        return salary;
    }
}

public class Main {
    public static void main(String[] args) {
        Employee e1 = new Employee("Ravi", 50000);
        Employee e2 = new Employee("Amit", 60000);

        System.out.println(e1.name);
        System.out.println(e1.getSalary());
        System.out.println(Employee.count);
        System.out.println(e1.company);
    }
}

```

Output:

```

Ravi
50000.0
2
TechCorp

```

`count` is `static`, so both employees share it and it reads 2. Every other field belongs to one object.

`final` has three uses, and they are worth separating:

```

final variable → cannot be reassigned
final method   → cannot be overridden
final class    → cannot be extended

```

2. Classes and inheritance

These build the type and connect it to others.

Keyword	What it does
<code>class</code>	Defines a blueprint for objects
<code>interface</code>	Defines a contract a class must follow
<code>abstract</code>	Defines a class that cannot be instantiated or a method without a body
<code>extends</code>	Inherits from a parent class
<code>implements</code>	Makes a class follow an interface
<code>super</code>	Refers to the parent class
<code>this</code>	Refers to the current object

First the contract and the base class:

```
interface Workable {
    void work();
}

abstract class Person {
    protected String name;

    Person(String name) {
        this.name = name;
    }

    abstract void introduce();

    void sleep() {
        System.out.println(name + " is sleeping");
    }
}
```

Then the class that joins them:

```

class Developer extends Person implements Workable {
    private String language;

    Developer(String name, String language) {
        super(name);           // calls the Person constructor
        this.language = language;
    }

    @Override
    void introduce() {
        System.out.println(name + " writes " + language);
    }

    @Override
    public void work() {
        System.out.println(name + " is writing code");
    }
}

```

And the program:

```

public class Main {
    public static void main(String[] args) {
        Developer dev = new Developer("Priya", "Java");

        dev.introduce();
        dev.work();
        dev.sleep();
    }
}

```

Output:

```

Priya writes Java
Priya is writing code
Priya is sleeping

```

`Developer` never wrote `sleep()`. It came from `Person` through `extends`.

```

Workable (interface)
    ↑ implements
Developer
    ↑ extends
Person (abstract)

```

A class can extend one class and implement many interfaces.

3. Control flow

These decide what runs next.

Keyword	What it does
<code>if</code> , <code>else</code>	Runs a block when a condition is true
<code>switch</code> , <code>case</code>	Chooses one branch from several values
<code>default</code>	Runs when no <code>case</code> matched
<code>for</code> , <code>while</code> , <code>do</code>	Repeats a block while a condition is true
<code>break</code>	Exits the loop or the <code>switch</code>
<code>continue</code>	Skips to the next turn of the loop
<code>return</code>	Exits the method, with a value if it has one

```
public class Main {
    public static void main(String[] args) {
        int[] scores = {45, 78, 92, 30, 65};

        for (int score : scores) {
            if (score < 40) {
                continue;           // skip this one, keep looping
            }
            if (score > 90) {
                System.out.println(score + ": excellent");
                break;              // stop the loop completely
            }
            System.out.println(score + ": pass");
        }

        int day = 3;
        switch (day) {
            case 1:
                System.out.println("Monday");
                break;
            case 3:
                System.out.println("Wednesday");
                break;
            default:
                System.out.println("Another day");
        }
    }
}
```

Output:

```
45: pass
78: pass
92: excellent
Wednesday
```

The loop never reaches 30 or 65, because `break` ended it at 92.

```
continue → skip this turn
break    → leave the loop
return   → leave the method
```

A missing `break` in a `switch` makes it fall into the next `case`, which is a common bug.

4. Exception handling

These deal with what goes wrong.

Keyword	What it does
<code>try</code>	Wraps code that might throw an exception
<code>catch</code>	Handles an exception of a given type
<code>finally</code>	Runs whether or not an exception was thrown
<code>throw</code>	Manually throws an exception
<code>throws</code>	Declares exceptions a method may throw

```

public class Main {

    static double divide(int a, int b) throws ArithmeticException {
        if (b == 0) {
            throw new ArithmeticException("Cannot divide by zero");
        }
        return (double) a / b;
    }

    public static void main(String[] args) {
        try {
            System.out.println(divide(10, 2));
            System.out.println(divide(5, 0));
        } catch (ArithmeticException e) {
            System.out.println("Caught: " + e.getMessage());
        } finally {
            System.out.println("Always runs");
        }
    }
}

```

Output:

```

5.0
Caught: Cannot divide by zero
Always runs

```

The second `divide` never printed. `throw` jumped straight to the `catch`. `finally` ran anyway, which is what makes it the right place to close a file or a connection.

```

throw → throws an exception now
throws → warns the caller that one may come

```

5. Types and checks

These name the kind of data.

Keyword	What it does
<code>int</code>	Holds a whole number
<code>long</code>	Holds a larger whole number
<code>double</code>	Holds a decimal number
<code>boolean</code>	Holds <code>true</code> or <code>false</code>

Keyword	What it does
<code>char</code>	Holds a single character
<code>void</code>	Indicates that a method returns no value
<code>instanceof</code>	Checks whether an object is an instance of a type
<code>enum</code>	Declares a fixed set of named values

```
enum Status {
    ACTIVE, INACTIVE, PENDING
}

public class Main {
    public static void main(String[] args) {
        int count = 10;
        long distance = 9876543210L;
        double price = 199.99;
        boolean active = true;
        char grade = 'A';

        System.out.println(count + " " + distance + " " + price
            + " " + active + " " + grade);

        Object value = "Hello";
        if (value instanceof String) {
            String text = (String) value;
            System.out.println("Length: " + text.length());
        }

        Status status = Status.PENDING;
        System.out.println(status);
    }
}
```

Output:

```
10 9876543210 199.99 true A
Length: 5
PENDING
```

`long` needs the `L` at the end, otherwise the compiler reads the number as an `int` and rejects it for being too large.

6. Threads and storage

These appear less often in everyday code, and interviews ask about them.

Keyword	What it does
<code>synchronized</code>	Allows one thread at a time into the method or block
<code>volatile</code>	Forces every thread to read the field from main memory
<code>transient</code>	Prevents a field from being serialized

```
class Counter {
    private int count = 0;

    synchronized void increment() {
        count++;
    }

    int get() {
        return count;
    }
}

public class Main {
    public static void main(String[] args) throws InterruptedException {
        Counter counter = new Counter();

        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) counter.increment();
        });
        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) counter.increment();
        });

        t1.start();
        t2.start();
        t1.join();
        t2.join();

        System.out.println(counter.get());
    }
}
```

Output:

```
2000
```

Without `synchronized` the answer is usually lower than 2000, because `count++` is three steps and two threads can read the same value before either writes.

`volatile` and `transient` are short enough to show without a whole program:

```
private volatile boolean running = true;    // one thread writes, another reads

private transient String password;          // skipped during serialization
```

7. Organizing code

Keyword	What it does
<code>package</code>	Declares which package the file belongs to
<code>import</code>	Brings in a type from another package

```
package com.example.payroll;

import java.util.List;
import java.util.ArrayList;

public class Payroll {
    private List<String> employees = new ArrayList<>();
}
```

`package` must be the first line in the file. Types in `java.lang`, such as `String` and `System`, need no import.

Words that are easy to confuse

These look alike	What it does
<code>final</code>	Stops something from being changed
<code>finally</code>	Always runs after <code>try</code>
<code>finalize</code>	An old cleanup method. Deprecated, do not use it
<code>throw</code>	Manually throws an exception
<code>throws</code>	Declares exceptions a method may throw
<code>this</code>	Refers to the current object
<code>super</code>	Refers to the parent class
<code>break</code>	Exits the loop
<code>continue</code>	Skips one turn of the loop
<code>static</code>	Makes a member belong to the class
<i>(no keyword)</i>	Makes a member belong to the object

Two more that interviews ask about.

`static` methods cannot use `this`, because there is no object. They can only reach other `static` members.

A `final` field must be given a value exactly once, either where it is declared or in the constructor.

The groups together

Group	Keywords
Access and object behavior	<code>public</code> , <code>private</code> , <code>protected</code> , <code>static</code> , <code>final</code> , <code>new</code>
Classes and inheritance	<code>class</code> , <code>interface</code> , <code>abstract</code> , <code>extends</code> , <code>implements</code> , <code>super</code> , <code>this</code>
Control flow	<code>if</code> , <code>else</code> , <code>switch</code> , <code>case</code> , <code>default</code> , <code>for</code> , <code>while</code> , <code>do</code> , <code>break</code> , <code>continue</code> , <code>return</code>
Exception handling	<code>try</code> , <code>catch</code> , <code>finally</code> , <code>throw</code> , <code>throws</code>
Types and checks	<code>int</code> , <code>long</code> , <code>double</code> , <code>boolean</code> , <code>char</code> , <code>void</code> , <code>instanceof</code> , <code>enum</code>
Threads and storage	<code>synchronized</code> , <code>volatile</code> , <code>transient</code>
Organizing code	<code>package</code> , <code>import</code>

Two words that look like keywords are not. `true`, `false` and `null` are literals, and `var` is a reserved type name that can still be used as a variable name.

The most useful thing at this point is not to memorize the list. Open any class from a real project and name the group each keyword in it belongs to.